Lab – Attaching to Docker Containers

To Begin with the Lab

Summary

In this lab, we learn how to attach ourselves to running Docker containers using the docker attach command. We explore how to run containers using different Docker images, including Nginx and Ubuntu.

We also understand why some containers automatically exit after starting, how to run an interactive Ubuntu container using the -it flag, and how to access the Ubuntu terminal running inside a Docker container.

Additionally, we learn the difference between the host Ubuntu terminal (WSL) and Ubuntu running inside a Docker container. Finally, we use Ctrl + P followed by Ctrl + Q to detach from a container without stopping it.

Understanding Docker Attach

The docker attach command allows us to connect our terminal to the main process running inside an existing container.

It is useful when:

- We want to view the output generated by a container.
- We want to connect to an interactive container.
- We want to observe the process running inside a container.

Syntax:

docker attach <container-id>

Example:

docker attach abc

Here, abc represents the first few characters of the container ID.

Docker allows us to use a unique shortened container ID instead of typing the complete ID.

Important: The docker attach command does not automatically open a new shell inside the container. It attaches to the container's main running process.

Understanding Interactive Mode (-it)

The -it flag is used to run a container in interactive mode with a pseudo-terminal.

It combines two options:

Flag	Full Form	Description
-i	Interactive	Keeps standard input open so we can interact with the container.
-t	TTY	Allocates a pseudo-terminal for terminal interaction.

Example:

docker run -it ubuntu

This command creates an Ubuntu container and opens an interactive terminal session.

Lab Steps

Step 1: Open Ubuntu Terminal

• Open Docker Desktop.

• Open your Ubuntu terminal (WSL).

• Verify Docker installation:

```
docker --version
```



• Check available Docker images:

```
docker images
```



Step 2: Run an Nginx Container

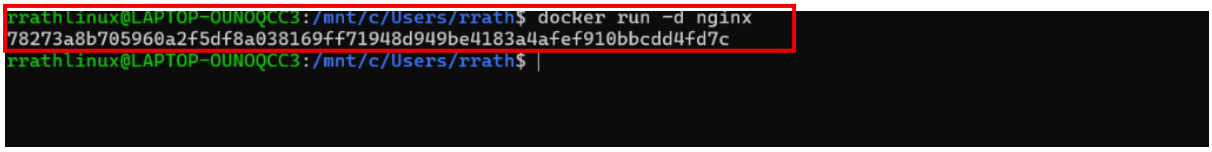
• Execute the following command:

```
docker run -d nginx
```

• Docker creates a new container using the Nginx image.

• Nginx starts running inside the container.

• Docker returns the container ID.



Step 3: Attach to the Nginx Container

• First, check running containers:

```
docker ps
```

• Copy the container ID.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker run -d nginx
78273a8b705960a2f5df8a038169ff71948d949be4183a4afef910bbcdd4fd7c
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              PORTS     NAMES
78273a8b7059   nginx     "/docker-entrypoint.…"   About a minute ago   Up About a minute   80/tcp    nervous_johnson
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

• Execute:

```
docker attach <container-id>
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker attach 78273a8b7059
^C2026/11/09 15:53:33 [notice] 1#1: signal 2 (SIGINT) received, exiting
2026/11/09 15:53:33 [notice] 34#34: exiting
2026/11/09 15:53:33 [notice] 29#29: exiting
2026/11/09 15:53:33 [notice] 30#30: exiting
2026/11/09 15:53:33 [notice] 32#32: exiting
2026/11/09 15:53:33 [notice] 33#33: exiting
2026/11/09 15:53:33 [notice] 36#36: exiting
2026/11/09 15:53:33 [notice] 36#36: exit
2026/11/09 15:53:33 [notice] 29#29: exit
2026/11/09 15:53:33 [notice] 31#31: exiting
2026/11/09 15:53:33 [notice] 31#31: exit
2026/11/09 15:53:33 [notice] 32#32: exit
2026/11/09 15:53:33 [notice] 30#30: exit
2026/11/09 15:53:33 [notice] 33#33: exit
2026/11/09 15:53:33 [notice] 35#35: exiting
2026/11/09 15:53:33 [notice] 35#35: exit
2026/11/09 15:53:33 [notice] 37#37: exiting
2026/11/09 15:53:33 [notice] 38#38: exiting
2026/11/09 15:53:33 [notice] 38#38: exit
2026/11/09 15:53:33 [notice] 34#34: exit
2026/11/09 15:53:33 [notice] 40#40: exiting
2026/11/09 15:53:33 [notice] 37#37: exit
2026/11/09 15:53:33 [notice] 40#40: exit
2026/11/09 15:53:33 [notice] 39#39: exiting
2026/11/09 15:53:33 [notice] 39#39: exit
2026/11/09 15:53:33 [notice] 1#1: signal 17 (SIGCHLD) received from 38
2026/11/09 15:53:33 [notice] 1#1: worker process 38 exited with code 0
2026/11/09 15:53:33 [notice] 1#1: signal 29 (SIGIO) received
2026/11/09 15:53:33 [notice] 1#1: signal 17 (SIGCHLD) received from 30
2026/11/09 15:53:33 [notice] 1#1: worker process 30 exited with code 0
2026/11/09 15:53:33 [notice] 1#1: signal 29 (SIGIO) received
2026/11/09 15:53:33 [notice] 1#1: signal 17 (SIGCHLD) received from 39
2026/11/09 15:53:33 [notice] 1#1: worker process 29 exited with code 0
2026/11/09 15:53:33 [notice] 1#1: worker process 31 exited with code 0
2026/11/09 15:53:33 [notice] 1#1: worker process 32 exited with code 0
```

Example:

docker attach abc

• Docker attaches your terminal to the container's main process.

• You may see Nginx log messages if the process generates output.

Observation:

You may feel that the terminal is stuck or not responding.

This happens because your terminal is attached to the Nginx process, which is running in the background and does not provide an interactive shell.

You cannot directly execute normal Linux commands in this attached session.

Step 4: Open Another Ubuntu Terminal

• Open another Ubuntu terminal window.

• Execute:

```
docker ps -a
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS                          PORTS     NAMES
78273a8b7059   nginx     "/docker-entrypoint.…"   6 minutes ago   Exited (0) About a minute ago             nervous_johnson
```

• You can see that the Nginx container is still running.

• Stop the container:

```
docker stop <container-id>
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker stop 78273a8b7059
78273a8b7059
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

The attached process in the first terminal will exit when the container stops.

Step 5: Download the Ubuntu Image

Now we will work with another Docker image.

• Execute:

```
docker pull ubuntu
```

• Docker downloads the official Ubuntu image from Docker Hub.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
f3f25b84a7d7: Pull complete
edafa32f5d9e: Pull complete
21594acc2d8b: Download complete
Digest: sha256:6a06111c3b0a8600feadb270aec771dc239bf407b25625f93f44ab00e11f4d3c
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

• Verify the image:

```
docker images
```

You should see the Ubuntu image in the list.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker images
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:latest   165ddde8751e        243MB         66.8MB    U
ubuntu:latest  6a06111c3b0a        163MB         45.9MB
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

Note: The docker run command automatically downloads an image if it is not available locally.

Step 6: Run an Ubuntu Container

• Execute:

```
docker run ubuntu
```

• Docker creates a container using the Ubuntu image.

• The container starts and then exits automatically.

• Check running containers:

```
docker ps
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker run ubuntu
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

• Check all containers:

```
docker ps -a
```

You will see the Ubuntu container with an Exited status.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED             STATUS                         PORTS     NAMES
f3a81838ba0f   ubuntu    "/bin/bash"              About a minute ago  Exited (0) About a minute ago            blissful_carver
78273a8b7059   nginx     "/docker-entrypoint.…"   13 minutes ago      Exited (0) 8 minutes ago                 nervous_johnson
```

Why?

The Ubuntu image does not automatically start a long-running interactive process. Once its default process finishes, the container exits.

Step 7: Try Running Ubuntu in Detached Mode

• Execute:

```
docker run -d ubuntu
```

• Docker creates a new Ubuntu container in detached mode.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker run -d ubuntu
d083063e95caab2b534b274f8fa30ccf652652ca39c9b6a5cb9a523c16d70742
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

• Check the container:

```
docker ps
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker run -d ubuntu
66b2ac37a1aa0860fcabc6c819c5103c06471505ae8832d1be950bad91eb566c
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

Observation:

The container exits again.

Reason: The -d flag only runs a container in the background. It does not create a process that keeps the container running.

Step 8: Run Ubuntu Using Interactive Mode

Now we will use the -it flag.

• Execute:

```
docker run -d -it ubuntu
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker run -d -it ubuntu
c1fcbc740f04a8bbdcfd4c819f6d68472679a6001005d89ea52fc9542064e5c7
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

Explanation:

Option Description

-d Runs the container in the background.

-i Keeps standard input open.

-t Allocates a pseudo-terminal.

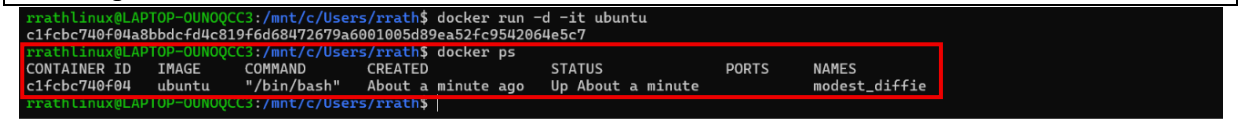
ubuntu Specifies the Ubuntu image.

• Docker creates an Ubuntu container.

• The interactive terminal remains available inside the container.

• Check the running container:

```
docker ps
```



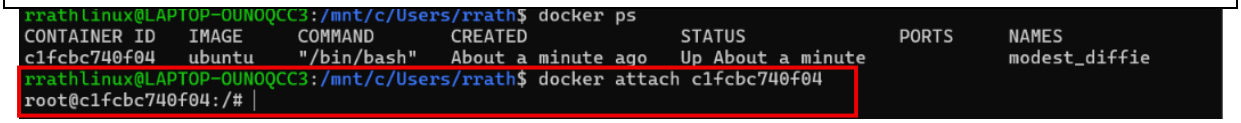
You should now see the Ubuntu container in the running state.

Step 9: Attach to the Ubuntu Container

• Copy the container ID.

• Execute:

```
docker attach <container-id>
```



Example:

docker attach abc

• You will now be attached to the Ubuntu container.

• You should see a terminal prompt similar to:

root@container-id:/#

Observation:

You are now working as the root user inside the Ubuntu container.

You are not working in the Ubuntu environment installed on your Windows machine through WSL.

You are working inside a separate Ubuntu environment running in a Docker container.

Step 10: Execute Linux Commands Inside the Container

Once attached, you can execute Linux commands.

1. Display directory contents

ls

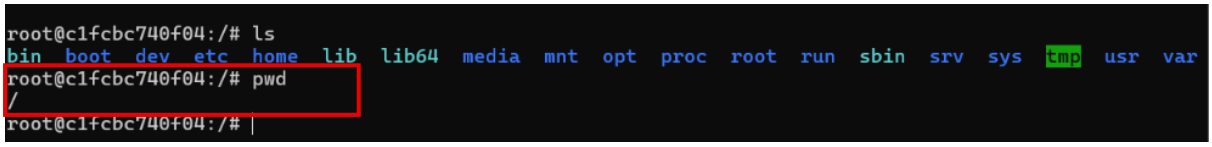
Displays files and directories in the current location.



2. Display current directory

pwd

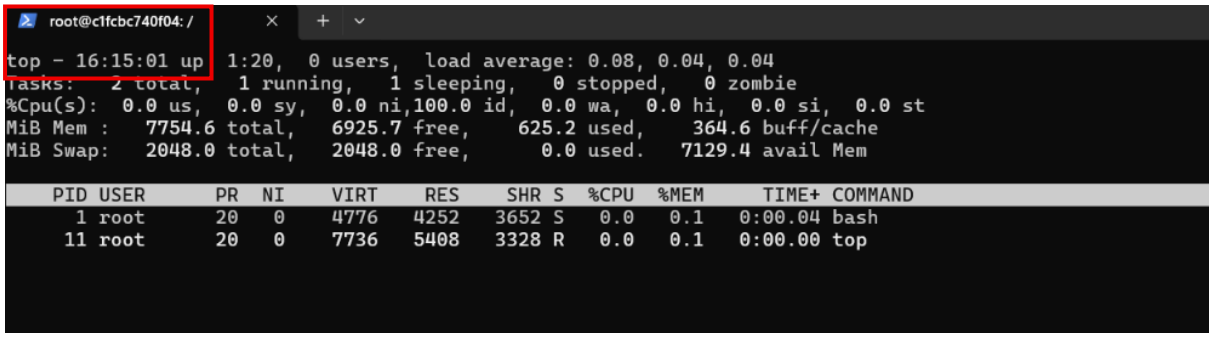
Displays the current working directory.



3. Check running processes

top

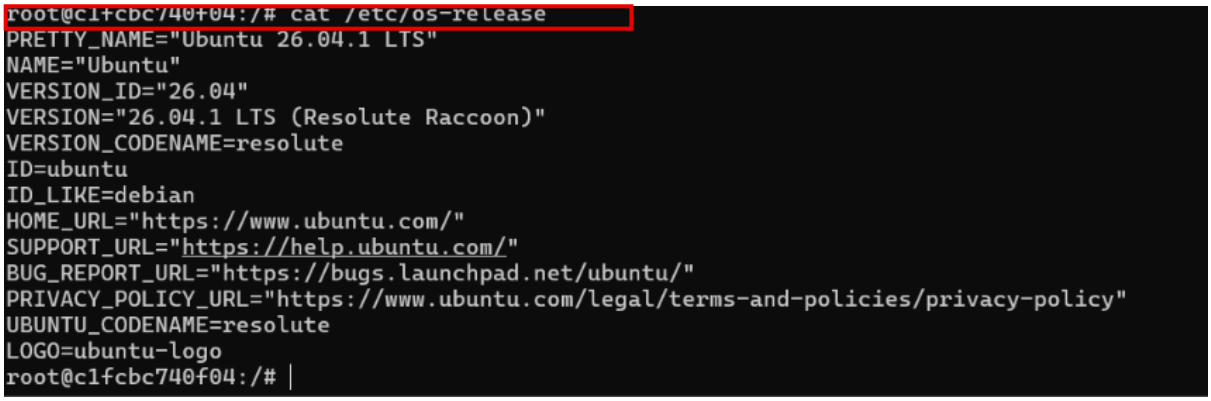
Displays currently running processes.



4. Display Ubuntu information

cat /etc/os-release

Displays information about the Ubuntu distribution.



5. Display current user

whoami

```
root@c1fcbc740f04:/# whoami
root
root@c1fcbc740f04:/#
```

Output:

root

These commands demonstrate that we can work with a Linux environment inside a Docker container.

Step 11: Detach from the Container

After executing Linux commands, we can return to our original Ubuntu terminal without stopping the container.

• Press:

Ctrl + P

• Then press:

Ctrl + Q

Observation:

- You will return to your original Ubuntu terminal.
- The Docker container will continue running.
- The container will not be stopped.

Step 12: Verify Container Status

• Execute:

docker ps

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS          PORTS     NAMES
c1fcbc740f04   ubuntu    "/bin/bash"   13 minutes ago   Up 13 minutes             modest_diffie
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

• You should see the Ubuntu container still running.

• You can also open Docker Desktop.

• Navigate to Containers.

• Verify that your Ubuntu container is in the running state.